

DSA Lab exercise 7

Submitted By: 25MCB1010, Patsamatla Bhaavya Ramita

Problem Statement 1: Matrix Chain Multiplication using Brute Force Approach

Given a sequence of matrices $A_1, A_2, \dots, A_n$ where A_i is of dimension $p_{i-1} \times p_i$, the objective is to determine all possible ways to fully parenthesize the matrix product and compute the number of scalar multiplications required for each parenthesization. Each valid parenthesization represents a specific way of grouping the matrix products, affecting the computational cost. The goal is to use Brute Force Approach to:

1. Generate all possible full parenthesizations of the matrix product.
2. Compute the scalar multiplication cost for each full parenthesization.
3. Print the total number of valid parenthesizations along with their corresponding multiplication costs.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>
#include <string.h>
#define MAX 10
typedef struct {
    char expr[200];
    int cost;
} Result;
int count = 0;
Result matrixChain(int p[], int i, int j) {
    Result res;
    if (i == j) {
        sprintf(res.expr, "A%d", i);
        res.cost = 0;
        return res; }
    res.cost = INT_MAX;
    for (int k = i; k < j; k++) {
        Result left = matrixChain(p, i, k);
```

```

        Result right = matrixChain(p, k + 1, j);
        int cost = left.cost + right.cost + p[i - 1] * p[k] * p[j];
        if (cost < res.cost) {
            res.cost = cost;
            sprintf(res.expr, "(%s*%s)", left.expr, right.expr); }
        printf("(%s*%s)\n%d\n", left.expr, right.expr, cost);
        count++; }
    return res; }

int main() {
    int n;
    printf("Enter number of matrices: ");
    scanf("%d", &n);
    int p[n + 1];
    printf("Enter %d dimensions: ", n + 1);
    for (int i = 0; i <= n; i++) {
        scanf("%d", &p[i]);
    }
    Result final = matrixChain(p, 1, n);
    printf("\nTotal number of fully parenthesized expressions: %d\n", count);
    return 0; }

```

Output:

```

Enter number of matrices: 4
Enter 5 dimensions: 10 20 30 40 50
(A3*A4)
60000
(A2*(A3*A4))
90000
(A2*A3)
24000
((A2*A3)*A4)
64000
(A1*((A2*A3)*A4))
74000
(A1*A2)
6000
(A3*A4)
60000
((A1*A2)*(A3*A4))
81000
(A2*A3)
24000
(A1*(A2*A3))
32000
(A1*A2)
6000
((A1*A2)*A3)
18000
(((A1*A2)*A3)*A4)
38000
Total number of fully parenthesized expressions: 13

```

Problem Statement 2: Matrix Chain Multiplication using Dynamic Programming.

Given a sequence of matrices $A_1, A_2, \dots, A_n$, where A_i is of dimension $p_{i-1} \times p_i$, the objective is to determine the optimal way to fully parenthesize the product such that the total number of scalar multiplications is minimized. Each valid parenthesization represents a specific way of grouping the matrix products, affecting the computational cost. The goal is to use Dynamic Programming to find the optimal order of multiplication, minimizing the number of scalar multiplications. Your task is to:

1. Determine the minimum number of scalar multiplications required.
2. Find the optimal parenthesization of the matrix chain that achieves this minimum cost.

Code:

```
#include <stdio.h>
#include <limits.h>
#include <string.h>
#define MAX 100
int m[MAX][MAX], s[MAX][MAX];
void printOptimalParens(int i, int j) {
    if (i == j) {
        printf("A%d", i);
        return;
    }
    printf("(");
    printOptimalParens(i, s[i][j]);
    printf("*");
    printOptimalParens(s[i][j] + 1, j);
    printf(")");
}
int main() {
    int n;
    printf("Enter number of matrices: ");
    scanf("%d", &n);
    int p[n + 1];
    printf("Enter %d dimensions: ", n + 1);
    for (int i = 0; i <= n; i++) {
        scanf("%d", &p[i]);
```

```

    }
    for (int i = 1; i <= n; i++) {
        m[i][i] = 0;
    }
    for (int L = 2; L <= n; L++) {
        for (int i = 1; i <= n - L + 1; i++) {
            int j = i + L - 1;
            m[i][j] = INT_MAX;
            for (int k = i; k < j; k++) {
                int cost = m[i][k] + m[k + 1][j] + p[i - 1] * p[k] * p[j];
                if (cost < m[i][j]) {
                    m[i][j] = cost;
                    s[i][j] = k;
                }
            }
        }
    }
    printf("Minimum scalar multiplications: %d\n", m[1][n]);
    printf("Optimal fully parenthesized expression: ");
    printOptimalParens(1, n);
    printf("\n");
    return 0;
}

```

Output:

```

Enter number of matrices: 3
Enter 4 dimensions: 10 20 30 40
Minimum scalar multiplications: 18000
Optimal fully parenthesized expression: ((A1*A2)*A3)
PS C:\Users\Bhaavya\Desktop\DSA\LabTask7>

```

Problem Statement 3: You are presented with a directed graph that consists of 6 vertices and 6 edges. Each edge in the graph carries a distinct capacity value, which denotes the maximum flow that the edge can accommodate. Your primary objective is to apply the Ford-Fulkerson

algorithm, a fundamental method in network flow theory, to determine the highest achievable flow from a designated source vertex to a target sink vertex within the graph.

Code:

```
#include <stdio.h>
#include <string.h>
#include <limits.h>
#define V 6
int capacity[V][V], flow[V][V], parent[V], visited[V];

int bfs(int s, int t) {
    memset(visited, 0, sizeof(visited));
    int queue[V], front = 0, rear = 0;
    queue[rear++] = s;
    visited[s] = 1;
    parent[s] = -1;

    while (front < rear) {
        int u = queue[front++];
        for (int v = 0; v < V; v++) {
            if (!visited[v] && capacity[u][v] - flow[u][v] > 0) {
                queue[rear++] = v;
                parent[v] = u;
                visited[v] = 1;
                if (v == t) return 1;
            }
        }
    }
    return 0;
}

int fordFulkerson(int s, int t) {
    int maxFlow = 0;
    memset(flow, 0, sizeof(flow));
```

```

while (bfs(s, t)) {
    int path_flow = INT_MAX;
    for (int v = t; v != s; v = parent[v]) {
        int u = parent[v];
        if (capacity[u][v] - flow[u][v] < path_flow)
            path_flow = capacity[u][v] - flow[u][v];
    }

    for (int v = t; v != s; v = parent[v]) {
        int u = parent[v];
        flow[u][v] += path_flow;
        flow[v][u] -= path_flow;
    }

    maxFlow += path_flow;
}

return maxFlow;
}

int main() {
    printf("Enter the adjacency matrix:");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &capacity[i][j]);
        }
    }
    int source, sink;
    printf("Enter source and sink vertices: ");
    scanf("%d %d", &source, &sink);
    int result = fordFulkerson(source, sink);
    printf("The maximum possible flow is %d\n", result);
    return 0;
}

```

Output:

```
Enter the adjacency matrix:
0 16 13 0 0 0
0 0 10 12 0 0
0 4 0 0 14 0
0 0 9 0 0 20
0 0 0 7 0 4
0 0 0 0 0 0
Enter source and sink vertices: 0 5
The maximum possible flow is 23
PS C:\Users\Bhaavya\Desktop\DSA\LabTask7>
```

Problem Statement 4: In a bustling metropolis, a city-wide network infrastructure upgrade is underway to ensure efficient communication between different devices and support the increasing demands of modern technology. As part of this upgrade, engineers are tasked with optimizing the flow of data through the communication network to maximize its capacity and reliability. You are a network engineer responsible for optimizing the data flow in the city's communication network. Your goal is to calculate the maximum amount of data that can be transferred through the network to ensure efficient communication between different devices, using the maximum flow problem.

Code:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <limits.h>
#define MAX 10
int capacity[MAX][MAX], flow[MAX][MAX], parent[MAX];
int visited[MAX], v;
int bfs(int s, int t) {
    memset(visited, 0, sizeof(visited));
    int queue[MAX], front = 0, rear = 0;
    queue[rear++] = s;
    visited[s] = 1;
    parent[s] = -1;
```

```

while (front < rear) {
    int u = queue[front++];
    for (int i = 0; i < v; i++) {
        if (!visited[i] && capacity[u][i] - flow[u][i] > 0) {
            queue[rear++] = i;
            parent[i] = u;
            visited[i] = 1;
            if (i == t) return 1;
        }
    }
}
return 0;
}

int maxFlow(int s, int t) {
    int total = 0;
    while (bfs(s, t)) {
        int path_flow = INT_MAX;
        for (int i = t; i != s; i = parent[i]) {
            int u = parent[i];
            if (capacity[u][i] - flow[u][i] < path_flow)
                path_flow = capacity[u][i] - flow[u][i];
        }
        for (int i = t; i != s; i = parent[i]) {
            int u = parent[i];
            flow[u][i] += path_flow;
            flow[i][u] -= path_flow;
        }
        total += path_flow;
    }
    return total;
}

int main() {
    int e;
    printf("Enter number of vertices and edges: ");

```

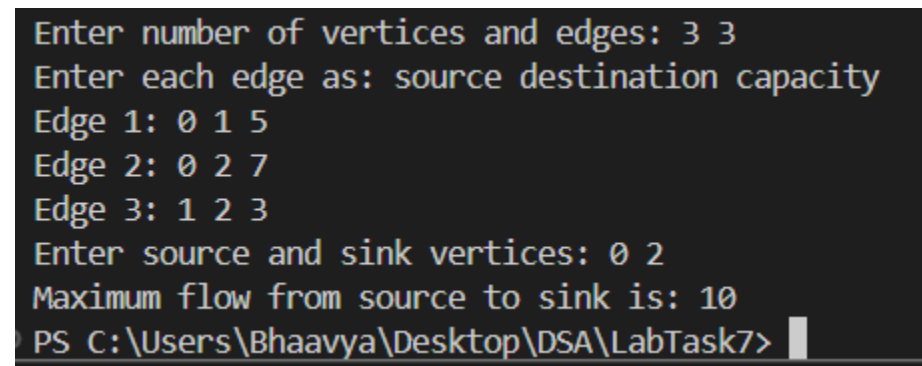


```

scanf("%d %d", &v, &e);
memset(capacity, 0, sizeof(capacity));
memset(flow, 0, sizeof(flow));
printf("Enter each edge as: source destination capacity\n");
for (int i = 0; i < e; i++) {
    int u, v_, c;
    printf("Edge %d: ", i + 1);
    scanf("%d %d %d", &u, &v_, &c);
    capacity[u][v_] = c;
}
int s, t;
printf("Enter source and sink vertices: ");
scanf("%d %d", &s, &t);
int result = maxFlow(s, t);
printf("Maximum flow from source to sink is: %d\n", result);
return 0;
}

```

Output:



```

Enter number of vertices and edges: 3 3
Enter each edge as: source destination capacity
Edge 1: 0 1 5
Edge 2: 0 2 7
Edge 3: 1 2 3
Enter source and sink vertices: 0 2
Maximum flow from source to sink is: 10
PS C:\Users\Bhaavya\Desktop\DSA\LabTask7>

```

Problem Statement 5: A bioinformatics researcher is studying DNA sequences to identify genetic similarities between different species. Given two DNA sequences, the researcher needs to determine the longest sequence of nucleotides (A, T, C, G) that appears in both sequences while maintaining the original order. Write a memoized recursive algorithm to compute the Longest Common Subsequence (LCS) of the given DNA sequences. **Output:** Function should output the LCS Sequence and its Length

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX 1000
int dp[MAX][MAX];
char lcs[MAX];

int LCS(char *X, char *Y, int m, int n) {
    if (m == 0 || n == 0) return 0;
    if (dp[m][n] != -1) return dp[m][n];
    if (X[m - 1] == Y[n - 1])
        return dp[m][n] = 1 + LCS(X, Y, m - 1, n - 1);
    int a = LCS(X, Y, m - 1, n);
    int b = LCS(X, Y, m, n - 1);
    return dp[m][n] = a > b ? a : b;
}

void buildLCS(char *X, char *Y, int m, int n) {
    int i = m, j = n, idx = dp[m][n];
    lcs[idx] = '\0';
    while (i > 0 && j > 0) {
        if (X[i - 1] == Y[j - 1]) {
            lcs[--idx] = X[i - 1];
            i--; j--;
        } else if (dp[i - 1][j] > dp[i][j - 1])
            i--;
        else
            j--;
    }
}

int main() {
    int m, n;
    printf("Enter the length of first sequence: ");
```

```

scanf("%d", &m);
char S1[m];
printf("Enter the first sequence: ");
for(int i=0; i<m; i++) {
    scanf(" %c", &S1[i]);
}
printf("Enter the length of second sequence: ");
scanf("%d", &n);
char S2[n];
printf("Enter the second sequence: ");
for(int i=0; i<n; i++) {
    scanf(" %c", &S2[i]);
}
memset(dp, -1, sizeof(dp));
int length = LCS(S1, S2, m, n);
buildLCS(S1, S2, m, n);
printf("LCS: %s\nLength: %d\n", lcs, length);
char X[] = "ACGTGCA";
char Y[] = "ACTGC";
m = strlen(X); n = strlen(Y);
memset(dp, -1, sizeof(dp));
length = LCS(X, Y, m, n);
buildLCS(X, Y, m, n);
printf("LCS: %s\nLength: %d\n", lcs, length);
return 0;}

```

Output:

```

Enter the length of first sequence: 29
Enter the first sequence: ACCGGTCGAGTGCGCGGAAGCCGGCCGAA
Enter the length of second sequence: 28
Enter the second sequence: GTCGTTCGGAATGCCGTTGCTCTGTAA
LCS: GTCGTTCGGAAGCCGGCCGAA
Length: 20
LCS: ACTGC
Length: 5

```

Problem Statement 6: Write a program to insert the following integers in the AVL tree.

4, 5, 7, 9, 10, 2, 3, 1

Code:

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
    int height;
};
int getHeight(struct Node* node) {
    if (node == NULL)
        return 0;
    return node->height;
}
int getMax(int a, int b) {
    return (a > b) ? a : b;
}
struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;
    newNode->height = 1;
    return newNode;
}
struct Node* rightRotate(struct Node* y) {
    struct Node* x = y->left;
    struct Node* temp = x->right;
    x->right = y;
    y->left = temp;
    y->height = getMax(getHeight(y->left), getHeight(y->right)) + 1;
    x->height = getMax(getHeight(x->left), getHeight(x->right)) + 1;
```

```

    return x;
}
struct Node* leftRotate(struct Node* x) {
    struct Node* y = x->right;
    struct Node* temp = y->left;
    y->left = x;
    x->right = temp;
    x->height = getMax(getHeight(x->left), getHeight(x->right)) + 1;
    y->height = getMax(getHeight(y->left), getHeight(y->right)) + 1;
    return y;
}
int getBalance(struct Node* node) {
    if (node == NULL)
        return 0;
    return getHeight(node->left) - getHeight(node->right);
}
struct Node* insertNode(struct Node* root, int value) {
    if (root == NULL)
        return createNode(value);
    if (value < root->data)
        root->left = insertNode(root->left, value);
    else if (value > root->data)
        root->right = insertNode(root->right, value);
    else
        return root;
    root->height = 1 + getMax(getHeight(root->left), getHeight(root->right));
    int balance = getBalance(root);

    if (balance > 1 && value < root->left->data)
        return rightRotate(root);
    if (balance < -1 && value > root->right->data)
        return leftRotate(root);
    if (balance > 1 && value > root->left->data) {
        root->left = leftRotate(root->left);
        return rightRotate(root);
    }
}

```

```

    if (balance < -1 && value < root->right->data) {
        root->right = rightRotate(root->right);
        return leftRotate(root);
    }
    return root;
}

void inorderTraversal(struct Node* root) {
    if (root != NULL) {
        inorderTraversal(root->left);
        printf("%d ", root->data);
        inorderTraversal(root->right);
    }
}

int main() {
    struct Node* avlRoot = NULL;
    int values[] = {4, 5, 7, 9, 10, 2, 3, 1};
    int totalValues = sizeof(values) / sizeof(values[0]);
    for (int i = 0; i < totalValues; i++) {
        avlRoot = insertNode(avlRoot, values[i]);
    }
    printf("Inorder traversal of the constructed AVL Tree:\n");
    inorderTraversal(avlRoot);
    return 0;
}

```

Output:

```

● Inorder traversal of the constructed AVL Tree:
  1 2 3 4 5 7 9 10
○ PS C:\Users\Bhaavya\Desktop\DSA\LabTask7>

```